02 de 20 - T2 24/02/2026 - T1 26/02/2026

Ambiente de desenvolvimento

PDF auxiliar no Classroom

Resumão

Comentários

// o que estiver a frente barra-barra é ignorado

```
/*
   esse aqui
   é um comentário
   de várias linhas
*/
```

```
/// <summary>
/// Realiza a soma de dois valores recebidos por parâmetros e retorna seu
resultado
/// </summary>
/// <param name="valor1">Primeiro valor inteiro</param>
/// <param name="valor2">Segundo  valor inteiro</param>
/// <returns>Soma dos dois valores recebidos por parâmetro</returns>
public static int Somar(int valor1, int valor2)
{
    return valor1 + valor2;
}
```

Quem já quiser conhecer um pouco mais, pode ver a documentação oficial em:
https://docs.microsoft.com/pt-br/dotnet/csharp/language-reference/xmldoc/recommended-tags

Identificadores - Regras

https://docs.microsoft.com/pt-br/dotnet/csharp/fundamentals/coding-style/identifier-names
https://docs.microsoft.com/pt-br/dotnet/csharp/language-reference/language-specification/lexical-structure#identifiers

- Os identificadores devem **começar** com uma **letra** ou **_**.
- Podem conter caracteres de letra **Unicode**, caracteres de dígito decimal, caracteres de conexão Unicode, caracteres de combinação Unicode ou caracteres de formatação Unicode.
- Por padrão, **não** podem ser **palavras-chave** (palavras reservadas) da linguagem, como: if, class, for, while, public etc
  - Caso necessário, é possível declarar identificadores que correspondem às palavras-chave em C# usando o prefixo **@** no identificador. O **@** não faz parte do nome do identificador. Por exemplo, **@if** declara um identificador chamado if. Esses identificadores textuais são destinados principalmente para interoperabilidade com os identificadores declarados em outras linguagens.

- Os identificadores distinguem o tipo de **caixa das letras**, isto é, as maiúsculas são consideradas distintas das minúsculas, ou seja, é **case sensitive**.
- Isso significa que **fimDeArquivo** é um identificador diferente de **FimDeArquivo**

Palavras-chave C#
palavras reservadas

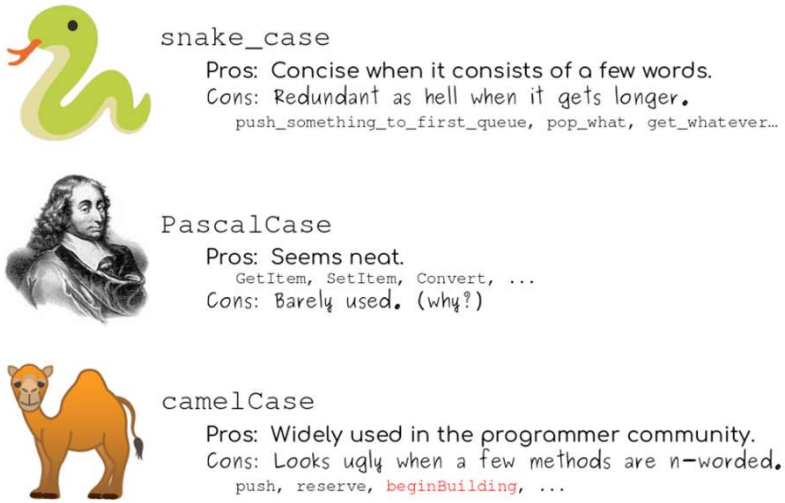
https://docs.microsoft.com/pt-br/dotnet/csharp/language-reference/keywords/

'abstract'	'as'	'base'	'bool'	'break'
'byte'	'case'	'catch'	'char'	'checked'
'class'	'const'	'continue'	'decimal'	'default'
'delegate'	'do'	'double'	'else'	'enum'
'event'	'explicit'	'extern'	'false'	'finally'
'fixed'	'float'	'for'	'foreach'	'goto'
'if'	'implicit'	'in'	'int'	'interface'
'internal'	'is'	'lock'	'long'	'namespace'
'new'	'null'	'object'	'operator'	'out'
'override'	'params'	'private'	'protected'	'public'
'readonly'	'ref'	'return'	'sbyte'	'sealed'
'short'	'sizeof'	'stackalloc'	'static'	'string'
'struct'	'switch'	'this'	'throw'	'true'
'try'	'typeof'	'uint'	'ulong'	'unchecked'
'unsafe'	'ushort'	'using'	'virtual'	'void'
'volatile'	'while'			

Pascal Case e Camel Case

- **camelCase** é a prática de escrever palavras compostas ou frases de modo que cada palavra ou abreviatura no meio da frase comece com uma letra maiúscula.

- Já **PascalCase** é a prática de escrever palavras compostas ou frases de modo que cada palavra ou abreviatura comece com uma letra maiúscula.



Convenções de nomenclatura

https://docs.microsoft.com/pt-br/dotnet/standard/design-guidelines/capitalization-conventions
https://learn.microsoft.com/pt-br/dotnet/csharp/fundamentals/coding-style/coding-conventions

- Quando codificamos é importante lembrar que seu código será lido por outros desenvolvedores da equipe e, por isso, é recomendável seguir padrões de nomenclatura.
- Além das regras, há muitas **convenções** de nomes para identificadores usadas em todas as APIs do .NET.
- A Convenção **PascalCase**, usada para **todos os identificadores** (Namespace, Type, Interface, Method, Property, Event, Field, Enum etc), exceto nomes de parâmetro, **coloca em maiúscula o primeiro caractere de cada palavra** (incluindo acrônimos sobre duas letras de comprimento), conforme mostrado nos exemplos a seguir:
  - public class **ExemploDeNomeDeClasse** { ... }
  - public string **ExemploNomeDeVariavel**;
  - public static readonly TimeSpan **OutroExemploNomeVariavel**;
  - namespace **MediaFinal { … }**
  - static void **Main**(string[] args) { … }
  - public interface **IEnumerable** { ... }
  - namespace **System.Security** { ... }

- Um caso especial é feito para acrônimos de duas letras nas quais as letras são colocadas em maiúsculas, conforme mostrado no identificador a seguir:
  - **IO**Stream

- A convenção **camelCase**, usada apenas para nomes de **parâmetros**, coloca em **maiúscula o primeiro caractere de cada palavra, exceto a primeira palavra**, conforme mostrado nos exemplos a seguir.
  - static void Main(string[] **args**) { … }
  - public string ExemploNomeDeMetodo(int **nomeParametro**) { ... }

- Além disso, as seguintes convenções são comuns:
  - Os nomes de **interface** começam com **I** maiúsculo.
  - Os tipos de atributo terminam com a palavra Attribute.
  - Os tipos enumerados usam um substantivo singular para não sinalizadores e um substantivo plural para sinalizadores.
  - Os identificadores não devem conter dois caracteres _ consecutivos. Esses nomes são reservados para identificadores gerados pelo compilador.

A tabela a seguir descreve as regras de capitalização para diferentes tipos de identificadores.

Identificador	Capitalização	Exemplo
Namespace	Pascal	`namespace System.Security { ... }`
Tipo	Pascal	`public class StreamReader { ... }`
Interface	Pascal	`public interface IEnumerable { ... }`

Método	Pascal	`public class Object {`
		`public virtual string ToString();`
		`}`

Propriedade	Pascal	`public class String {`
		`public int Length { get; }`
		`}`

Evento	Pascal	`public class Process {`
		`public event EventHandler Exited;`
		`}`

Campo	Pascal	`public class MessageQueue {`
		`public static readonly TimeSpan`
		`InfiniteTimeout;`
		`}`
		`public struct UInt32 {`
		`public const Min = 0;`
		`}`

Valor de enumeração	Pascal	`public enum FileMode {`
		`Append,`
		`...`
		`}`

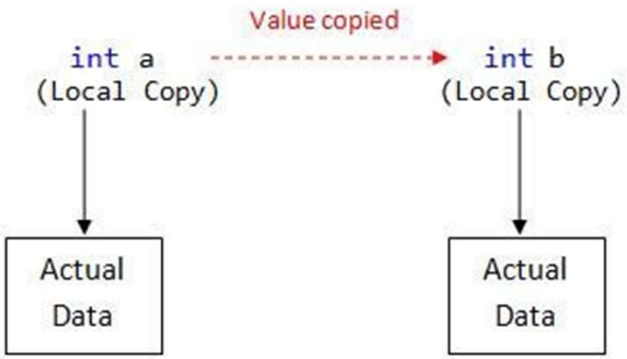
Parâmetro	Camel	`public class Convert {`
		`public static int ToInt32(string value);`
		`}`

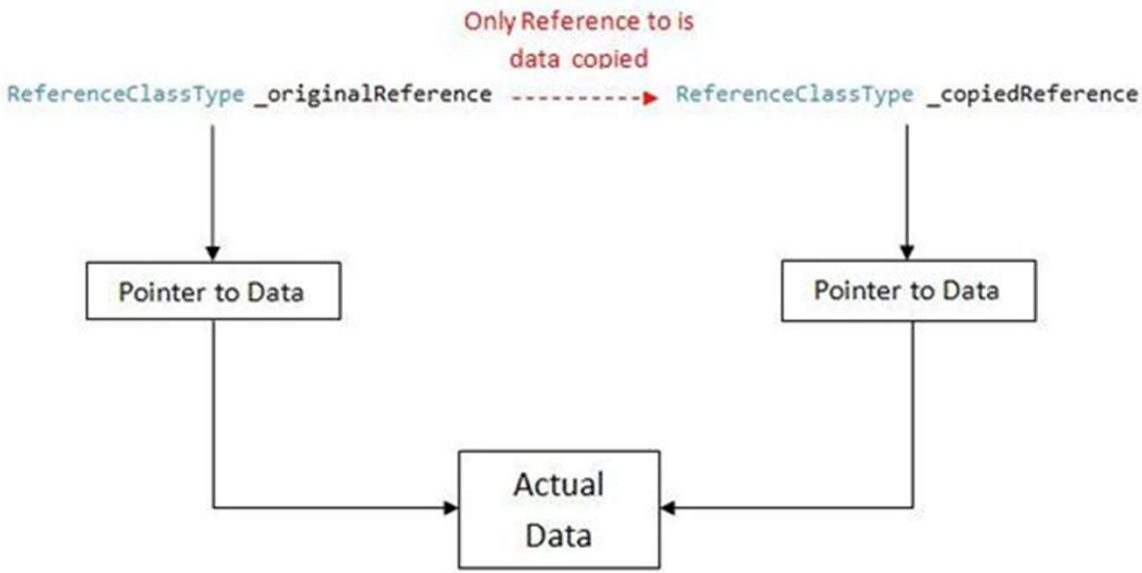
Tipos de dados

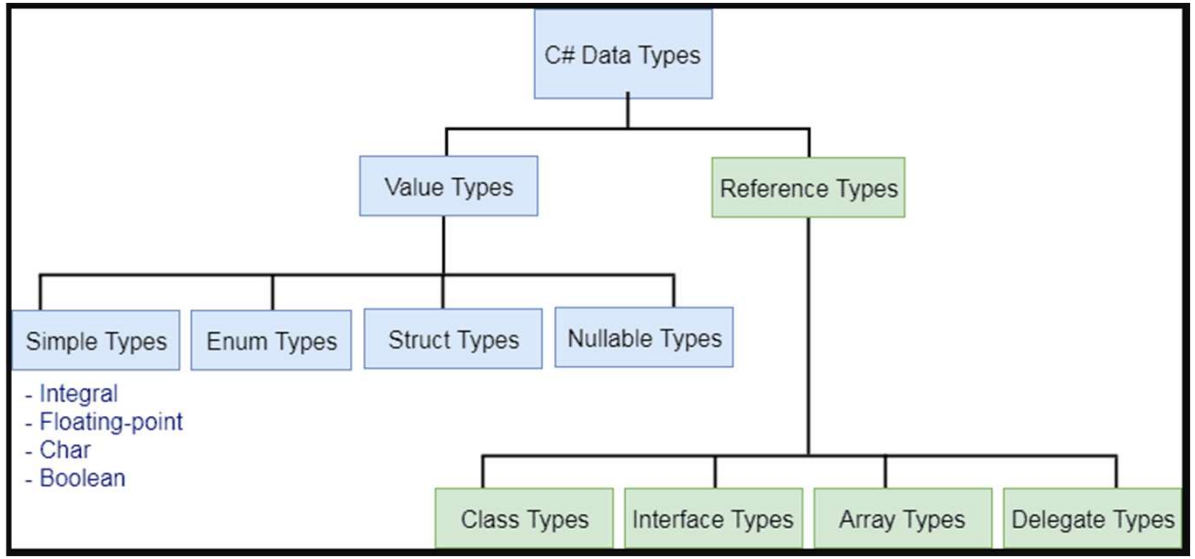
https://docs.microsoft.com/pt-br/dotnet/csharp/language-reference/builtin-types/value-types
https://docs.microsoft.com/pt-br/dotnet/csharp/language-reference/language-specification/types#value-types

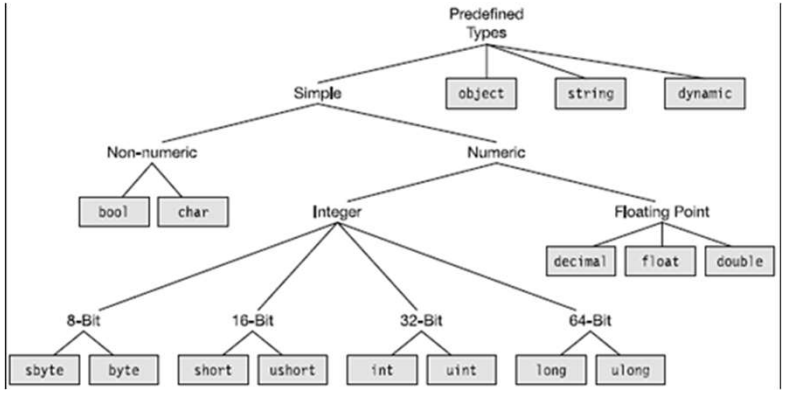
- C# é uma linguagem de programação **fortemente tipada**, isso significa que todas as variáveis e objetos devem ter um tipo declarado/definido.
- Os tipos de dados são divididos em **value types** e **reference types**.
- As variáveis **value type** contém dentro delas um **valor**, enquanto as **reference type** contém uma **referência**.
- Isso significa que se copiar uma variável do tipo **value type** para dentro de outra o **valor é copiado** e, se o mesmo for feito com uma do tipo **reference type** será copiado apenas a **referência do objeto**.
- Os **value types** são derivados de **System.ValueType** e **reference types** são derivados de **System.Object**.

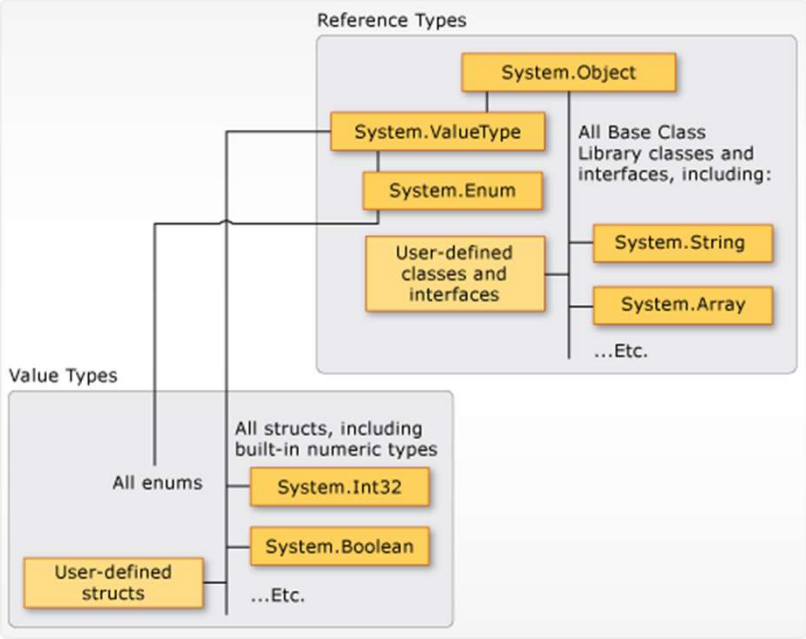
- O valor atribuído a uma variável deve estar de acordo com o seu tipo declarado.
- Por exemplo uma variável declarada com o tipo int não pode receber um valor numérico de ponto flutuante, de tipo decimal por exemplo.


Value copied
int a
(Local Copy)
int b
(Local Copy)
Actual Data
Actual Data


Only Reference to is data copied
ReferenceClassType _originalReference
ReferenceClassType _copiedReference
Pointer to Data
Pointer to Data
Actual Data


C# Data Types
Value Types
Reference Types
Simple Types
Enum Types
Struct Types
Nullable Types
- Integral
- Floating-point
- Char
- Boolean
Class Types
Interface Types
Array Types
Delegate Types


Predefined Types
Simple
object
string
dynamic
Non-numeric
Numeric
bool
char
Integer
Floating Point
decimal
float
double
8-Bit
16-Bit
32-Bit
64-Bit
sbyte
byte
short
ushort
int
uint
long
ulong


Reference Types
System.Object
System.ValueType
System.Enum
All Base Class Library classes and interfaces, including:
User-defined classes and interfaces
System.String
System.Array
...Etc.
Value Types
All structs, including built-in numeric types
All enums
System.Int32
System.Boolean
User-defined structs
...Etc.

Value types

- Um **value type** é um tipo de **struct** ou um tipo de **enumeração**.
- O **C#** fornece um **conjunto de tipos de struct predefinidos** chamados de **tipos simples**.
- Os **tipos simples** são identificados por meio de **palavras reservadas**.
- O C# fornece os seguintes **tipos de valor internos/incorporados**, também conhecidos como **tipos simples**:
  - Tipos numéricos integrais (**sbyte**, **byte**, **short**, **ushort**, **int**, **uint**, **long**, **ulong**, **char**)
  - Tipos numéricos de ponto flutuante (**float**, **double**)
  - Tipo de valor booleano (**bool**)
  - Tipos monetários (**decimal**)

- A partir do **C# 7,0**, temos também suporte a **tuplas** de valor. Uma **tupla de valor** é um **value type**, mas não é um tipo simples.

- As principais características dos **value types** são:
  - As variáveis que são baseadas em tipos por valor contêm **valores** e atribuir uma variável de por valor a outra, **copia o valor contido**.
  - Todos os tipos de valor são derivados implicitamente de **System.ValueType**;
  - Você pode usar literais para fornecer um valor de um tipo simples. Por exemplo, **'A'** é um literal do tipo **char** e **300** é um literal do tipo **int**.
  - Você pode declarar constantes dos tipos simples com a palavra-chave **const**. Expressões constantes, cujos operandos são todas constantes dos tipos simples, são avaliadas no momento da compilação.
  - Você não pode derivar um novo tipo de um tipo por valor.
  - Ao contrário dos tipos de referência, um tipo por valor não pode conter o valor **null**. No entanto, usar o recurso dos **tipos anuláveis** (**Nullable Types**) permite que tipos por valor sejam atribuídos a **null**.

value types - integral_type

- **sbyte**: inteiros de **8 bits signed (1 byte)** com valores entre **-128** e **127**.
- **byte**: inteiros de **8 bits unsigned (1 byte)** com valores entre **0** e **255**.
- **short**: inteiros de **16 bits signed (2 bytes)** com valores entre **-32768** e **32767**.
- **ushort**: inteiros de **16 bits unsigned (2 bytes)** com valores entre **0** e **65.535**.
- **int**: inteiros de **32 bits signed (4 bytes)** com valores entre **-2147483648** e **2147483647**.
- **uint**: inteiros de **32 bits unsigned (4 bytes)** com valores entre **0** e **4294967295**, sufixo **u**.
- **long**: inteiros de **64 bits signed (8 bytes)** com valores entre **-9223372036854775808** e **9223372036854775807** , sufixo **l**.
- **ulong**: inteiros de **64 bits unsigned (8 bytes)** com valores entre **0** e **18446744073709551615** , sufixo **ul**.

- **char**: **16 bits single Unicode character**. Any valid character, e.g. a,*, \x0058 (hex), or\u0058 (Unicode).
  - inteiros de **16 bits unsigned (2 bytes)** com valores entre **0** e **65535**. O conjunto de valores possíveis para o tipo char corresponde ao conjunto de **caracteres Unicode**.
  - As constantes do tipo char devem ser gravadas como **character_literal** ou como **integer_literal** em combinação com uma conversão para tipo char. Por exemplo, (char)10 é o mesmo que '\x000A'.

value types - floating_point_type

- **float**: **32-bit Single-precision floating point** type **(4 bytes)**, representar valores que variam de aproximadamente **1.5 * 10^-45 a 3.4 * 10^38** com uma **precisão de 7 dígitos** , sufixo **f**.

- **double**: **64-bit double-precision floating point** type **(8 bytes)**, representar valores que variam de aproximadamente **5.0 * 10^-324** a **1.7 × 10^308** com uma **precisão de 15-16 dígitos** , sufixo **d**.

- OBS: o separador de casas decimais padrão no código é o **.** (ponto).

value types - decimal

- **decimal**: é um tipo de dados de **128 bits (16 bytes)** adequado para **cálculos financeiros** e **monetários**. Pode representar valores variando de **1.0 * 10^-28** a aproximadamente **7.9 * 10^28** com **28-29 dígitos significativos** , sufixo **m**.

value types - bool

- **bool**: 8-bit logical true/false value **1 byte**, os únicos valores possíveis são **True** e **False**.
  - Nunca tente usar 0 ou 1, outras linguagens como c++ aceitam isto, mas C# não.

Alias vs .NET Type

- Os **tipos simples** quer vimos anteriormente são identificados por meio de **palavras reservadas**, mas essas palavras reservadas são simplesmente **aliases** para tipos de **struct** predefinidos no **namespace System**, conforme descrito na tabela a seguir.

Palavra reservada/Alias	.NET Type	Tipo
sbyte	System.Sbyte	struct
byte	System.Byte	struct
short	System.Int16	struct
ushort	System.UInt16	struct
int	System.Int32	struct
uint	System.UInt32	struct
long	System.Int64	struct
ulong	System.UInt64	struct
char	System.Char	struct
float	System.Single	struct
double	System.Double	struct
bool	System.Boolean	struct
decimal	System.Decimal	struct

- Por exemplo, **int** é um alias de **System.Int32**, então toda vez que você escreve **int**, o compilador traduz o código para o tipo **System.Int32**.

- Então, como um **tipo simples** é **alias** de **um tipo struct**, cada tipo simples tem **membros**.

- Por exemplo, **int** tem os membros declarados em **System.Int32** e os membros herdados de **System.Object**, e isso permite realizar algumas operações, por exemplo:

```
int max = int.MaxValue;          // System.Int32.MaxValue constant
int min = int.MinValue;          // System.Int32.MinValue constant
string s = i.ToString();         // System.Int32.ToString() instance method
string t = 123.ToString();       // System.Int32.ToString() instance method
```

Valores padrão

- Cada **tipo** de dado possui um **valor padrão**. O tipo **numérico** é **0**, **booleano** tem **falso** e **char** tem **'\0'** como valor padrão.
- Use **default(typename)** para atribuir um **valor padrão do tipo de dados** ou do C# 7.1 em diante, use o literal **default**.

```
int i = default(int);          // 0
float f = default(float);      // 0
decimal d = default(decimal);          // 0
bool b = default(bool);        // false
char c = default(char);        // '\0'

// C# 7.1 onwards
int i = default;               // 0
float f = default;             // 0
decimal d = default;           // 0
bool b = default;              // false
char c = default;              // '\0'
```

Tipos de tupla

- O C# oferece suporte a **tuplas**, que fornece sintaxe concisa para agrupar vários elementos de dados em uma estrutura de dados leve. Você cria uma instância de uma tupla declarando os tipos e os nomes dos membros entre (e) , conforme mostrado no exemplo a seguir:

```
(string Nome, int Idade) pessoa = ("Luciano", 18);
Console.WriteLine($"{pessoa.Nome}, você tem {pessoa.Idade} anos.");
// Output:
// Luciano, você tem 18 anos.
```

- As tuplas fornecem uma alternativa para a estrutura de dados com vários membros, sem usar os blocos de construção.

- https://docs.microsoft.com/pt-br/dotnet/csharp/language-reference/builtin-types/value-tuples

Tipos Nulos (Nullable Types)

- Ao contrário de uma variável de um tipo de referência, uma variável de um **tipo de valor** pode conter o valor null somente se o tipo de valor for um tipo anulável.
- Para cada tipo de valor não anulável, há um tipo de valor anulável correspondente, indicando o mesmo conjunto de valores, mais o valor **null**.
- A atribuição a uma variável de um tipo de valor cria uma cópia do valor que está sendo atribuído. Isso difere da atribuição a uma variável de um tipo de referência, que copia a referência, mas não o objeto identificado pela referência.
- Com exceção da variável do tipo string o restante dos tipos podem ser declarados para receber valores nulos, úteis em operações envolvendo banco de dados ou em situações que podem apresentar valores nulos inesperados, para isto basta acrescentar uma interrogação ao final da declaração, ficando assim int?, long?, float?, double?, decimal?, char? e bool?.

```
int? meuNumero = null;
```

Variáveis Reference Type

- As variáveis **reference type** armazenam apenas a **referência do objeto**. Os tipos de referência são:
  - Tipo **object**: é a classe base de todos os outros tipos. Todo tipo em C#, direta ou indiretamente, deriva de object. A palavra-chave object é simplesmente um alias para a classe predefinida System.Object.
  - Tipo **dinâmico**: como object, pode fazer referência a qualquer objeto. Quando os operadores são aplicados a expressões do tipo dynamic, sua resolução é adiada até que o programa seja executado.
  - **DateTime:** Represents date and time 0:00:00am 1/1/01 to 11:59:59pm 12/31/9999.

  - Tipo **string** (cadeia de caracteres)**:** é um tipo de classe herdado diretamente do object. As instâncias de string representam as **cadeias de caracteres Unicode**. Os valores do tipo string **podem ser gravados como literais de cadeia de caracteres**. A palavra-chave **string** é simplesmente um alias para a classe predefinida **System.String**. É utilizado para se armazenar caracteres e o valor de uma string deve estar entre aspas duplas.

```
string nome = "Uniplac";
```

Alias vs .NET Type

- Os **tipos** quer vimos anteriormente são identificados por meio de **palavras reservadas**, mas essas palavras reservadas são simplesmente **aliases** para tipos predefinidos no **namespace System**, conforme descrito na tabela a seguir.

Palavra reservada/Alias	**.NET Type**	**Tipo**
object	System.Object	Class
string	System.String	Class
DateTime	System.DateTime	struct

palavra chave **var**

https://docs.microsoft.com/pt-br/dotnet/csharp/language-reference/keywords/var

- As variáveis que são declaradas no escopo do método podem ter um tipo implícito **var**. Uma variável local de **tipo implícito** é **fortemente tipada**, como se você mesmo tivesse declarado o tipo, mas o compilador determina o tipo. As duas declarações a seguir são funcionalmente equivalentes:

```
int x = 0;        // definição explicita ou direta do tipo de dados da variável x
var y = 0 ;       // definição implícito ou indireta do tipo de dados da variável y

Console.WriteLine( x.GetType() );  // output: System.Int32
Console.WriteLine( y.GetType() );  // output: System.Int32
```

- A seguir temos alguns exemplos que mostram várias maneiras de declarar variáveis locais usando **var**:

```
var i = 5;                          // i é compilado como um int
var s = "Abc";                      //s é compilado como uma string
var c = 'A';                        //c é compilado como um char
var a = new[] {0,1,2};              //a é compilado como um int[]
var u = new {Nome = "abc", Idade = 34};   // u é compilado como um tipo anônimo
var lista = new List<int>();        // listra é compilada como um List<int>
```

vamos
codar

Variáveis – projeto - exemplo

- Veja como fica fácil utilizar isso em código.
- Crie um novo projeto chamado **MediaFinal**:
  - Abrir nova pasta com nome MediaFinal

dotnet new console

- De forma engessada mesmo, armazene 4 notas e o nome do aluno em variáveis.
- Mostre o nome do aluno e sua média final.

WARNING
SPOILER
ALERT

dotnet new console

```
internal class Program
{
    private static void Main(string[] args)
    {

        decimal nota1, nota2, nota3, nota4;
        string nome;

        nome = "Luciano Coelho";

        nota1 = 9.8m;
        nota2 = 9.2m;
        nota3 = 5.0m;
        nota4 = 8.9m;

        var media = (nota1 + nota2 + nota3 + nota4) / 4;

        Console.Write($"{nome}, sua média final é {media}");


    }
}
```

dotnet run

dotnet run --project ..\MediaFinal

Conversões **implícitas** ou **promoção**

- Suponha que temos um código que declara uma variável do tipo **int** (4 bytes) e depois tenta copiar seu conteúdo para uma variável **long** (8 bytes):

```
int valor = 1;
long valorGrande = valor;
```

- Nesse caso, mesmo sendo tipos diferentes, como o **tamanho** de uma variável **long** é maior do que o de uma variável **int**, o compilador sabe que podemos **copiar o seu conteúdo sem perder informações**, aplicando assim automaticamente a **conversão implícita**, também chamada de **promoção**.

Conversões **implícitas** ou **promoção**

- **Conversões implícitas ou promoção** acontecem quando **não há potencial perda de valores**, não sendo desta forma necessário informar ao compilador.
- Exemplos:

```
short aux1 = 10;

int aux2 = aux1;          //conversão de short para int

ushort aux3 = 'b';        //conversão de char para ushort – 98

float aux4 = aux1;        //conversão de short para float
```

Implicit Conversion From	To
sbyte	short, int, long, float, double, decimal
byte	short, ushort, int, uint, long, ulong, float, double, decimal
short	int, long, float, double, or decimal
ushort	int, uint, long, ulong, float, double, or decimal
int	long, float, double, or decimal.
uint	long, ulong, float, double, or decimal
long	float, double, or decimal
ulong	float, double, or decimal
char	ushort, int, uint, long, ulong, float, double, or decimal
float	Double

Conversões **explicitas** ou **casting**

- Agora vamos tentar copiar o valor de **int** (4 bytes) para uma variável do tipo **short** (4 bytes):

```
int valor2 = 1;
short valorPequeno = valor;
```

- Nesse código, tentamos copiar o conteúdo de uma variável maior para dentro de uma de tamanho menor. Essa cópia pode ser perigosa pois o valor que está na variável do tipo **int** pode não caber na variável **short** e, por isso, o compilador gera um erro de compilação quando tentamos fazer essa conversão.

Não é possível converter implicitamente tipo "int" em "short".

- Para forçarmos o compilador a fazer uma conversão perigosa, precisamos utilizar uma operação chamada **casting**, ou **conversão explicita**, falando para qual tipo queremos fazer a conversão.

```
int valor2 = 1;
short valorPequeno = (short) valor2;
```

Conversões **explicitas** ou **casting**

- **Conversões explicitas ou casting** são utilizadas nos casos onde uma perda de valores pode ocorrer, neste caso o programador tem que informar ao compilador que esta ciente desta possível perda.

```
long aux5 = 59;
byte aux6 = (byte) aux5;        //converte de long para byte
char aux7 = (char) 98;  //converte de ushort para char - b
int aux8  =  (int)  aux5;//converte de long para int
```

Conversões **explicitas** ou **casting**
Truncar

- Quando atribuímos um valor de um tipo **maior** para um tipo **menor**, como por exemplo, um valor do tipo **Integer** para um do tipo **Byte** o C# vai **truncar o que não puder ser suportado** pela variável de tipo **menor**.
- No caso de Integer (4 bytes) para Byte (1 byte) serão descartados os valores acima de 1 byte.

```
int aux9 = (int)5000.99;        //trunca o valor, pegando somente a parte inteira
byte aux10 = (byte)aux9;        //trunca o valor, 5000 é maior que 255, output 136
```

- **Observação:**
  - Para truncar é utilizada a função **mod**, exemplo:

    5000 mod 256

  - 5000 é representado como 4 bytes (int) (hexadecimal)

    00 00 13 88
  - Agora, quando você o converte em byte, leva apenas o último 1 byte.

    88
  - que é 136 na representação decimal

DE \ PARA	***byte***	***short***	***char***	***int***	***long***	***float***	***double***
byte		Implícito	char	Implícito	Implícito	Implícito	Implícito
short	byte		char	Implícito	Implícito	Implícito	Implícito
char	byte	short		Implícito	Implícito	Implícito	Implícito
int	byte	short	char		Implícito	Implícito	Implícito
long	byte	short	char	int		Implícito	Implícito
float	byte	short	char	int	long		Implícito
double	byte	short	char	int	long	float	

Conversoes
dotnet new console

```
internal class Program
{
    private static void Main(string[] args)
    {

        // Conversões implícitas ou promoção
        int valor = 1;
        long valorGrande = valor;   //conversão de int para long
        short aux1 = 10;
        int aux2 = aux1;    //conversão de short para int
        ushort aux3 = 'b';  //conversão de char para ushort - 98
        float aux4 = aux1; //conversão de short para float

        // Conversões explicitas ou casting
        int valor2 = 1;
        short valorPequeno = (short)valor2;
        long aux5 = 59;
        byte aux6 = (byte)aux5;     //converte de long para byte
        char aux7 = (char)98;   //converte de ushort para char - b
        int aux8 = (int)aux5;   //converte de long para int


        int aux9 = (int)5000.99;     //trunca o valor, pegando somente a parte inteira
        byte aux10 = (byte)aux9;    //trunca o valor, 5000 é maior que 255, output 136

    }
}
```

Convertendo string em número

https://docs.microsoft.com/pt-br/dotnet/csharp/programming-guide/types/how-to-convert-a-string-to-a-number

- Converta um **string** em um **número** chamando o método **Parse** ou **TryParse** encontrado em tipos numéricos (int, long, double e assim por diante).
- Você usa métodos **Parse** ou **TryParse** no tipo numérico que espera que a cadeia de caracteres contenha, como o tipo **System.Int32**.

- O método **Parse** retorna o número convertido;

```
int number = int.Parse("11");
ulong v8 = ulong.Parse("698988");
```

  - Se a cadeia de caracteres não estiver em um formato válido, o Parse lançará uma exceção.
  - Ao chamar um método **Parse**, você deve usar a manipulação de exceção para capturar um FormatException quando a operação de análise falhar.

Convertendo string em número

https://docs.microsoft.com/pt-br/dotnet/csharp/programming-guide/types/how-to-convert-a-string-to-a-number

- O método **TryParse** retorna um valor booleano que indica se a conversão foi bem-sucedida e retorna o número convertido em um parâmetro **out**.

```
if ( Int32.TryParse("11", out int j) ) {
    Console.WriteLine( j );
} else {
    Console.WriteLine("Não foi possível fazer a conversão.");
}
```

  - Se a cadeia de caracteres não estiver em um formato válido **TryParse** retornará false.

Observação
ponto flutuante

- Quando trabalhamos com **conversão de ponto flutuante**, temos o problema do caractere utilizado para separar as casas decimais, no Brasil utilizamos a **virgula** (,) e o padrão americano é o **ponto** (.).
- Nas etapas que envolve interação com o usuário, o C# realiza o tratamento de **Cultura**, na pratica, por padrão, a linguagem reconhece o **idioma do sistema operacional** onde a aplicação esta executando e aplica as formatações conforme o padrão dessa cultura, ou seja, em uma maquina com sistema operacional em Português Brasileiro, quando o usuário tiver que digitar um valor com casa decimal, deverá informar esse valor separado por virgula.

//float.Parse(String stringName, CultureInfo objectName);

```
float v14 = float.Parse("134.43", System.Globalization.CultureInfo.InvariantCulture.NumberFormat);
float v15 = float.Parse("134,43");
```

Cultura

- Nas etapas que envolve interação com o usuário, o **C#** realiza o tratamento de **Cultura**, na pratica, por padrão, a linguagem reconhece o **idioma do sistema operacional** onde a aplicação esta executando e aplica as formatações conforme o padrão dessa cultura, ou seja, em uma maquina com sistema operacional em Português Brasileiro, quando o usuário tiver que digitar um valor com casa decimal, deverá informar esse valor separado por virgula.
- Ou seja, quando trabalhamos com **conversão de ponto flutuante**, temos o problema do caractere utilizado para separar as casas decimais, no Brasil utilizamos a **virgula** (,) e o padrão americano é o **ponto** (.).

```
Console.WriteLine( CultureInfo.CurrentCulture );
Console.WriteLine( CultureInfo.CurrentUICulture );
pt-BR
pt-BR
```

```
//float.Parse(String stringName, CultureInfo objectName);
float v14 = float.Parse("134.43", System.Globalization.CultureInfo.InvariantCulture.NumberFormat);
float v15 = float.Parse("134,43");
```

- Diretamente no código
  - Sem tratamento de Cultura
  - Padrão americano, (.).

- Iteração com o usuário
  - Com tratamento de Cultura
  - Conforme a cultura utilizada, (,) virgula para pt-BR

ConverteStringNumero
dotnet new console

```
internal partial class Program
{
    private static void Main(string[] args)
    {
        // converte de string para números
        sbyte v1 = sbyte.Parse("1");
        byte v2 = byte.Parse("2");
        short v3 = short.Parse("3");
        ushort v4 = ushort.Parse("4");
        int v5 = int.Parse("5");
        uint v6 = uint.Parse("6");
        long v7 = long.Parse("7");
        ulong v8 = ulong.Parse("8");
        //com ponto flutuante cuidar a cultura ( . ou , )
        float v9 = float.Parse("9,99");
        double v10 = double.Parse("10.10", System.Globalization.CultureInfo.InvariantCulture.NumberFormat);
        decimal v11 = decimal.Parse("11,11");
        char v12 = char.Parse("c");
        bool v13 = bool.Parse("true");
        Console.WriteLine($"{v1} {v2} {v3} {v4} {v5} {v6} {v7} {v8} {v9} {v10} {v11} {v12} {v13} ");
        if (int.TryParse("-105", out int j))
        {
            Console.WriteLine(j);
        }
        else
        {
            Console.WriteLine("String could not be parsed.");
        }
        const string inputString = "abc";
        if (int.TryParse(inputString, out int numValue))
        {
            Console.WriteLine(numValue);
        }
        else
        {
            Console.WriteLine($"Int32.TryParse could not parse '{inputString}' to an int.");
        }
    }
}
```

vamos
codar

Variáveis – projeto - exemplo

- Pegue o exemplo anterior da **MediaFinal** e ajuste para que todos os valores sejam digitados pelo usuário.
- Para efeito didático, trate as 4 notas como números inteiros.
- O valor médio deve apresentar casas decimais.
- Tenha o cuidado para utilizar os tipos adequados nas variáveis.

- Mostre o nome do aluno e sua média final.

WARNING
SPOILER
ALERT

```
internal partial class Program
{
    private static void Main(string[] args)
    {
        byte nota1, nota2, nota3, nota4, soma;
        decimal media;
        string nome;

        //lê o valor no console e armazena em variáveis auxiliares
        //o valor lido do console, ou de uma tela, sempre será string
        Console.Write("Nome do aluno: ");
        string auxNome = Console.ReadLine();
        Console.Write("Valor da nota 1: ");
        string auxN1 = Console.ReadLine();
        Console.Write("Valor da nota 2: ");
        string auxN2 = Console.ReadLine();
        Console.Write("Valor da nota 3: ");
        string auxN3 = Console.ReadLine();
        Console.Write("Valor da nota 4: ");
        string auxN4 = Console.ReadLine();

        //atribui os valores auxiliares as variáveis do calculo
        nome = auxNome;
        //nos tipos numéricos é necessária conversão
        nota1 = byte.Parse(auxN1);
        nota2 = byte.Parse(auxN2);
        nota3 = byte.Parse(auxN3);
        nota4 = byte.Parse(auxN4);

        //necessário o casting, pois o compilador entende que o resultado é int
        soma = (byte)(nota1 + nota2 + nota3 + nota4);

        //se a divisão for somente de inteiros, o resultado será inteiro
        //aqui vamos usar o sufixo no 4 para indicar que é decimal
        media = soma / 4M;

        // uso de cadeias de caracteres interpoladas $ {} {} para mostrar o resultado
        Console.WriteLine($"{nome}, sua média final é: {media}");
    }
}
```

Operadores aritméticos

https://docs.microsoft.com/pt-br/dotnet/csharp/language-reference/operators/arithmetic-operators

- Os operadores a seguir executam operações aritméticas com operandos de tipos numéricos:
  - **Operadores binário**:

    ***** (multiplicação), **/** (divisão), **%** (restante), **+** (adição) e **-** (subtração)
  - **Operadores unários**:

    ++ (incremento), **--** (decrementar), **+** (além) e **-** (menos)

- Esses operadores têm suporte de todos os tipos numéricos integral e de ponto flutuante.

Operadores binário

- **Operadores binário**:
  ***** (multiplicação), **/** (divisão), **%** (restante), **+** (adição) e **-** (subtração)

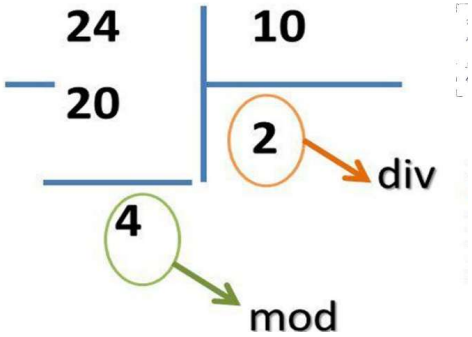
- Veja a alguns exemplos:

```
int quatro = 2 + 2;
int três = 5 – 2;
int oito = 4 * 2;
int dezesseis = 64 / 4;
```

Operadores especiais (mod e div)

- **x MOD y**
  - Retorna o **resto** da divisão entre 2 números.
  - Utiliza o caractere **%** como operador.

    int resultadoMod = 24 % 10;



- **x DIV y**
  - Retorna o **valor inteiro** que resulta da divisão entre 2 números.
  - Não possui um operador especifico na programação.
  - Utiliza **CAST** para pegar somente a parte inteira da divisão.

    int resultadoDiv = (int) 24 / 10;

Atribuição composta
+=, -=, *=, /=, %=

- Para um operador binário **op**, uma expressão de atribuição composta do formato
  x op= y que é equivalente a **x = x op y** exceto que **x** é avaliado apenas uma vez.

- O seguinte exemplo demonstra o uso da **atribuição composta** com **operadores aritméticos**:

```
int a = 5;

a += 9;
Console.WriteLine(a);  // output: 14

a -= 4;
Console.WriteLine(a);  // output: 10

a *= 2;
Console.WriteLine(a);  // output: 20

a /= 4;
Console.WriteLine(a);  // output: 5

a %= 3;
Console.WriteLine(a);  // output: 2
```

```
int a = 5;

a = a + 9;

a = a - 4;

a = a * 2;

a = a / 2;

a = a % 3;
```

Atribuição composta promoção

- Devido a **promoções numéricas**, o resultado da operação **op** pode não ser **implicitamente conversível no tipo T de x**.
- Nesse caso, se **op** for um **operador predefinido** e o **resultado da operação for explicitamente convertido no tipo T de x**, uma expressão de atribuição composta da forma **x op= y** será equivalente a **x = (T)(x op y)**, exceto que x será avaliada apenas uma vez, ou seja, será aplicado o **casting**, com perca de valor.
- O exemplo a seguir demonstra esse comportamento, ou seja o maior valor de um byte é **255**, e ao realizar a atribuição composta o valor seria **300**, sendo **truncado** o valor para poder ser atribuído na variável, ou seja, será atribuído um valor diferente do esperado!

```
byte x = 200;
byte y = 100;

var c = x + y;
Console.WriteLine( c.GetType() );  // output: System.Int32
Console.WriteLine( c );  // output: 300

// será aplicado o casting, com perca de valor.
x += y;
Console.WriteLine( x );  // output: 44
```

Operadores unários

- **Operadores unários:**

  ++ (incremento),

  -- (decremento),

  + (além, positivo unário - sem efeito)

  - (menos, negativo unário)

```
r = -w;      // r -10, w 10
r = +w;      // r 10, w 10
r = +r;      // r 10
```

- Veja a alguns exemplos:

```
int r = default, w = 10 ;    // r 0, w 10
r = w++;      // r 10, w 11
r = ++w;      // r 12, w 12
r = w--;      // r 12, w 11
r = --w;      // r 10, w 10
```

```
//
int val1 = 4, val2 = 10, val3;
val3 = val1++ + --val2; // 5, 9, 13
```

++	Incremento	x++	Retorna x e depois soma 1
		++x	Soma 1 antes de retornar x
--	Decremento	x--	Retorna x e depois subtrai 1
		--x	Subtrai 1 antes de retornar x
-	Negativo (unário)	-x	Altera o sinal de x
+	Positivo (unário)	+x	Sem efeito
*	Multiplicação	x * y	Multiplica x por y
/	Divisão	x / y	Divide x por y
%	Módulo	x % y	Resto da divisão de x por y
+	Adição	x + y	Adiciona x e y
-	Subtração	x – y	Subtrai y de x

Precedência e associatividade do operador

- A seguinte lista ordena os operadores aritméticos da precedência mais alta para a mais baixa:
  - Incluir um pós-fixo a operadores de incremento x++ e decremento x--
  - Incluir um prefixo a operadores de incremento ++x e de decremento --x e operadores unários + e -
  - Operadores de multiplicação *, / e %
  - Operadores de adição + e -
  - Operadores aritméticos binários são associativos à esquerda. Ou seja, os operadores com o mesmo nível de precedência são avaliados da esquerda para a direita.

- Use parênteses, (), para alterar a ordem de avaliação imposta pela precedência e pela capacidade de associação do operador.
  - Em algoritmos, colchetes e chaves não são empregados no lugar de parênteses

```
Console.WriteLine(2 + 2 * 2);   // output: 6
Console.WriteLine((2 + 2) * 2); // output: 8

Console.WriteLine(9 / 5 / 2);   // output: 0 - arredondando
Console.WriteLine(9 / (5 / 2)); // output: 4 - arredondando
```

LABORATÓRIO PRÁTICO
PARCIAL DE NOTA AVALIAÇÃO 02

- Realize a instalação e configuração do seu ambiente de desenvolvimento **.NET/C#**, incluindo o **.NET 10.0** e o **Visual Studio Code**.

- Conforme exemplificação, realize a criação do projeto **HelloWorld** e de **seu primeiro código completo**.
- Em seguida execute e veja se tudo correu bem.
- Caso tenha ocorrido erros, tente identificar e corrigir.

- Execute o programa e tire um **PRINT** pegando **toda a tela do VSCode**, devendo aparecer o **EXPLORADOR** com a estrutura de pastas e arquivos do projeto, **Program.cs** aberto com um **comentário contendo seu nome completo** e o **resultado da execução no Terminal**.
  - Veja um exemplo do PRINT na próxima pagina.

Arquivo Editar Seleção Ver Acessar ··· HelloWorld

EXPLORADOR

HELLOWORLD
- bin
- obj
- HelloWorld.csproj
- HelloWorld.sln
- Program.cs

Program.cs > Program

0 referências
```
internal class Program
{
    0 referências
    private static void Main(string[] args)
    {
        Console.WriteLine("Luciano Coelho!");

        DateTime dataCorrente = DateTime.Now;
        Console.WriteLine("Data/Hora: " + dataCorrente);

    }
}
```

PROBLEMAS SAÍDA TERMINAL PORTAS COMENTÁRIOS CONSOLE DE DEPURAÇÃO HelloWorld.exe

```
PS C:\POO\HelloWorld> dotnet build
Restauração concluída (0,3s)
  HelloWorld êxito (1,5s) → bin\Debug\net9.0\HelloWorld.dll

Construir êxito em 2,0s
PS C:\POO\HelloWorld> dotnet run
Luciano Coelho!
Data/Hora: 10/02/2025 11:24:19
PS C:\POO\HelloWorld>
```

ESTRUTURA DO CÓDIGO
LINHA DO TEMPO
SOLUTION EXPLORER

⊗ 0 ⚠ 0 Projetos: HelloWorld Debug Any CPU Live Share Ln 11, Col 2 Espaços: 4 UTF-8 with BOM CRLF {} C#

LABORATÓRIO PRÁTICO
PARCIAL DE NOTA AVALIAÇÃO 02

- Revise o material da aula, realizando a leitura da teorização aplicada.
- Após isso, realize as **7 primeiras etapas** deste treinamento introdutório disponibilizado pela Microsoft, não leva mais do que 30 minutos.
- Dê seus primeiros passos com o C#:

https://docs.microsoft.com/pt-br/learn/paths/csharp-first-steps/?WT.mc_id=dotnet-35129-website

- Na etapa de **Verificação de conhecimentos** tire um PRINT da tela com suas respostas.

LABORATÓRIO PRÁTICO
PARCIAL DE NOTA AVALIAÇÃO 02

Crie um programa chamado **NumerosSomaMultiplica** que solicite ao usuário para digitar dois números em ponto flutuante devendo somar os dois números e multiplicar o resultado pelo primeiro número.

Mostre o resultado conforme o exemplo:

O primeiro número lido foi 5,65, o segundo numero lido foi 7,99. O resultado final é: 77,06.

Execute o programa e tire um **PRINT** pegando **toda a tela do VSCode**, devendo aparecer o **EXPLORADOR** com a estrutura de pastas e arquivos do projeto, **Program.cs** aberto com um **comentário contendo seu nome completo** e o **resultado da execução no Terminal**.

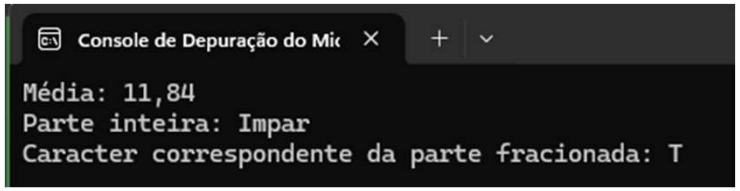
LABORATÓRIO PRÁTICO
PARCIAL DE NOTA AVALIAÇÃO 02

- Escreva um programa que realize o seguinte
  - Peça para o usuário digitar três valores decimais
    - Realize a validação se o valor digitado é valido ou não
    - Caso não seja, solicite que o usuário digite novamente
    - O programa somente deve avançar após ter três valores validos

  - Calcule a média dos valores digitados
    - O resultado deve conter, armazenado na variável, somente duas casas decimais
    - Não deve ser realizado arredondamento

  - Informe como saída
    - O valor médio
    - A indicação se a parte inteira do valor médio é par ou impar
    - O caractere correspondente da parte fracionada da média

  - Tudo deve ser organizado em métodos!!!



- Execute o programa e tire um **PRINT** pegando **toda a tela do VSCode**, devendo aparecer o **EXPLORADOR** com a estrutura de pastas e arquivos do projeto, **Program.cs** aberto com um **comentário contendo seu nome completo** e o **resultado da execução no Terminal**.

LABORATÓRIO PRÁTICO
PARCIAL DE NOTA AVALIAÇÃO 02

- Uma empresa concederá um aumento de salário aos seus funcionários, variável de acordo com o cargo, conforme a tabela abaixo.

Código	Cargo	Percentual
101	Gerente	10%
102	Engenheiro	20%
103	Técnico	30%

- Escreve um programa que leia o nome, salário e o cargo de um funcionário e calcule o novo salário.
- Se o cargo do funcionário não estiver na tabela, ele deverá, então, receber 40% de aumento.
- Mostre o nome, o salário antigo, o novo salário e a diferença.

- Utilize métodos!!!

LABORATÓRIO PRÁTICO
PARCIAL DE NOTA AVALIAÇÃO 02

Escreva um programa chamado **AgoraEuEntendo** onde o usuário **digita cinco valores** e então seja realizado o seguinte:

- O **primeiro valor** deve ser armazenado em uma variável do tipo **byte**
- O **segundo valor** deve ser armazenado em uma variável do tipo **short**
- O **terceiro valor** deve ser armazenado em uma variável do tipo **int**
- O **quarto valor** deve ser armazenado em uma variável do tipo **long**
- O **quinto valor** deve ser armazenado em uma variável do tipo **float**
- O **total** deve ser calculado e armazenado em uma variável do tipo **double**
- O **valor médio** deve ser calculado e armazenado em uma variável do tipo **decimal**
- Obtenha a **parte inteira** do valor médio e armazene em uma variável do tipo **char**

- Mostre no final o total, o valor médio, a parte inteira da média e o caractere gerado com a parte inteira da média. Exemplo de saída se forem entrados os seguintes valores (55,160,10,110,152):

  Soma: 487.0 Média: 97.4 Parte Inteira: 97 Carácter: a

Execute o programa e tire um **PRINT** pegando **toda a tela do VSCode**, devendo aparecer o **EXPLORADOR** com a estrutura de pastas e arquivos do projeto, **Program.cs** aberto com um **comentário contendo seu nome completo** e o **resultado da execução no Terminal**.

LABORATÓRIO PRÁTICO
PARCIAL DE NOTA AVALIAÇÃO 02

- Vocês ficaram com **6 laboratórios práticos** para realizarem e entregar.
- **Data de entrega**

  T2 – 24/02

  T1 – 26/02

- **Data entrega da recuperação**

  T2 – 03/03 18:40

  T1 – 05/03 18:40

- Deve ser postado na atividade correspondente do Classroom o seguinte:
  - Os 6 PRINTs solicitados nos exercícios anteriores.
  - Se eventualmente tiver problema e não conseguir concluir, deve ser entregue um arquivo com uma justificava contendo os motivos, reforço que os motivos devem ser técnicos.

- Os arquivos NÃO podem ser compactados, ou seja, poste no Classroom os 6 arquivos de forma individual!

Documentação Oficial
https://docs.microsoft.com/pt-br/learn/paths/csharp-first-steps/?WT.mc_id=dotnet-35129-website
https://msdn.microsoft.com
https://docs.microsoft.com/pt-br/dotnet/
https://docs.microsoft.com/pt-br/dotnet/csharp/whats-new/csharp-version-history
https://docs.microsoft.com/pt-br/dotnet/csharp/whats-new/csharp-9
https://social.msdn.microsoft.com/Forums/pt-br/home
https://code.visualstudio.com/docs/languages/csharp
https://docs.microsoft.com/pt-br/dotnet/core/tools/dotnet-new
https://docs.microsoft.com/pt-br/dotnet/standard/glossary

.NET In-Browser Tutorial
https://dotnet.microsoft.com/learn?WT.mc_id=dotnet-35129-website
https://www.microsoft.com/net/tutorials/csharp/getting-started

Visual Studio Code
https://code.visualstudio.com/
https://code.visualstudio.com/docs/languages/csharp
https://docs.microsoft.com/pt-br/dotnet/core/tutorials/with-visual-studio-code